Training a Neural Network to play Flappy bird

General Idea

The goal of this projet is to train a neural network in playing the game “flappy bird”. The game is a phone game and consists of a bird at the left edge of the screen, which can be commanded to jump by tapping the screen. Pairs of vertical “pipes” with gaps between them are moving right to left on the screen, and the bird needs to be controlled so that it moves through the gaps between the pipes.

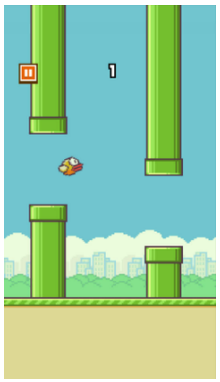

Figure 1: The original flappy bird game

To make this game playable by the AI, a basic “propagate(state, pressed)” function, which takes a game state and the information whether the player is tapping the screen, and returns a new game state, needs to be programmed. The AI model is given the current state and returns either a “yes, press the screen” or a “no, don’t press the screen” choice, which influenes the “pressed” boolean given to the propagate function.

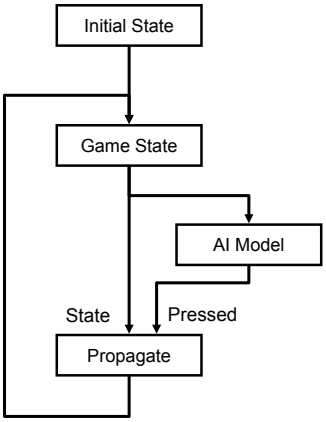

Figure 2: Game loop used to let the model play a single game of flappy bird

To train the AI, we used a evolutionary training approach. We first create a collection of models with random weights and let each model play a game. Then, we rank the models by their respective score and only keep a sample of the best performing models. Then we apply a method to create “children” from the set of the best models, and permutate the children (“mutation”).

The mutated children then play the game again and the next generation is evaluated.

Over time, the best performers of each generation will make up the population of the next generation, increasing the general performance, while the mutation ensures that changes that increase performance are tried.

Implementation of the game

The game consists of the functions “propagate(state, pressed)”, “get_initial_state()” and the helper function “get_height()”.

propagate(state, pressed)

The propagate function simulates a single step in time of the game. It takes the state of the previous step and the decision whether the screen is tapped, and return the state after the time step.

The state

The state is a dictionary containing values that describe the current conditions of the game. The values are:

Value	Description
height	Current height of the bird, from 0 (bottom) to 1 (top)
speed	Speed at which the pipes are approaching the bird
vert_vel	Vertical velocity of the bird (positive is up, negative is down) in height units per step
next_dist	Distance to the next pair of pipes. Initialized to 1 every time the current pair is passed
next_height	Height of the center of the gap of the next pair of pipes
frame	Current frame (time step) count
last_input	Frame number when last tap was registered (used to implement tap cooldown so button can not be held down).

Only the first 5 values (height to next_height) are given to the model to make the press/don’t press decision.

get_initial_state()

The “get_initial_state()”-Function is used to get the state when the game is started. The bird starts in the middle of the screen, the first pair of pipes is 1 unit away and the vertical velocity is 0. The height of the gap is set to a random value, using the “get_height()” function.

get_height()

The “get_height()”-Function returns a random value for the height of the gap between the pipes. The random value is uniformly distributed in a range centered around the middle of the screen (height 0.5) and with a span set in the game settings.

Model Implementation

General model properties

The model is implemented using torch. We chose a neural network with 5 input neurons, a hidden layer with 7 neurons and 2 output neurons. The activation function between the layers is the ReLu function. The 2 output neurons are converted to a boolean using the softmax function.

Creating child generations

After evaluating the current generation by letting each model play one game, we need to create children for the next generation.

cloning

A simple way (called "cloning" here) is to pick a subset of the best models, and copy them to get the same number of children as there were parents. If there are 100 parents, one could take the 5 best models and take 20 copies of each as the children, for example. Taking a copy from multiple of the best models instead of just the single best model ensures that a model that got lucky but is generally worse at playing the game doesn't completely take over the gene pool.

crossover_pick

A different way to create the children is called "crossover_pick" here. With this method, we create the same number of child models as there were models in the generation before. Then, we randomly select 2 parents from the n best models of the parent generation for each child. The parents are chosen completely independently so both parents can actually be the same model. Then, for each weight of the child model, we randomly pick one of the two parents and use the weight of the selected parent in the child.

crossover_avg

For this method, we create childdren and select 2 parents per child in the same way as with "crossover_pick". Once the parents are determined, the child network weights are calculated by simply taking the average of the weights of the parents.

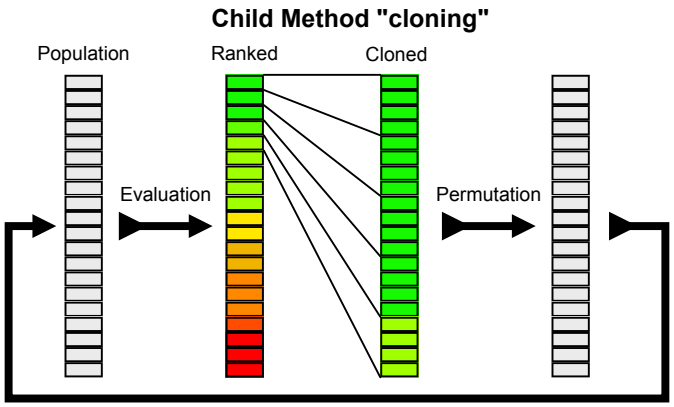

Figure 3: Training process using the child method "cloning"

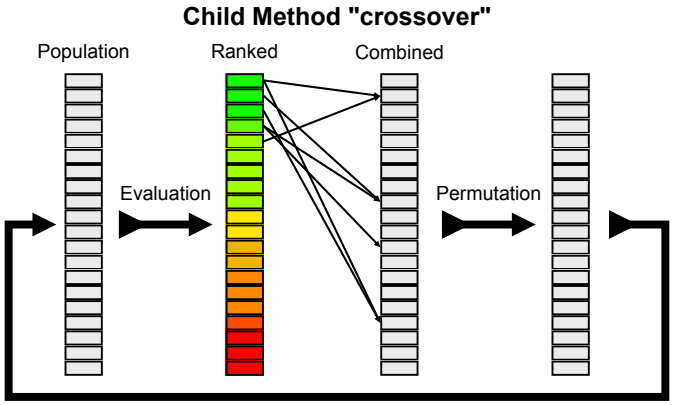

Figure 4: Training process using the child method “crossover_pick” or “crossover_avg”

Parameter Variation

The parameters that can be varied in the training process are mutation rate and the “child method”. To evaluate which parameters give the best result, we performed a parameter variation study. For this, we ran 10 trainings per parameter set (to average out random variations) for each combination of parameter ranges. The parameter choices for the mutation rate were 0.01, 0.1, 0.2 and 0.5. The parameter choices for the child method were “cloning”, “crossover_pick” and “crossover_avg”. We trained 40 generations per training run. This results in $n_{\text{run\_per\_param\_set}} * n_{\text{generations\_per\_run}} * n_{\text{mutation\_rate\_choices}} * n_{\text{child\_method\_choices}} = 10 * 40 * 3 * 3 = 3600$ Generations to train.

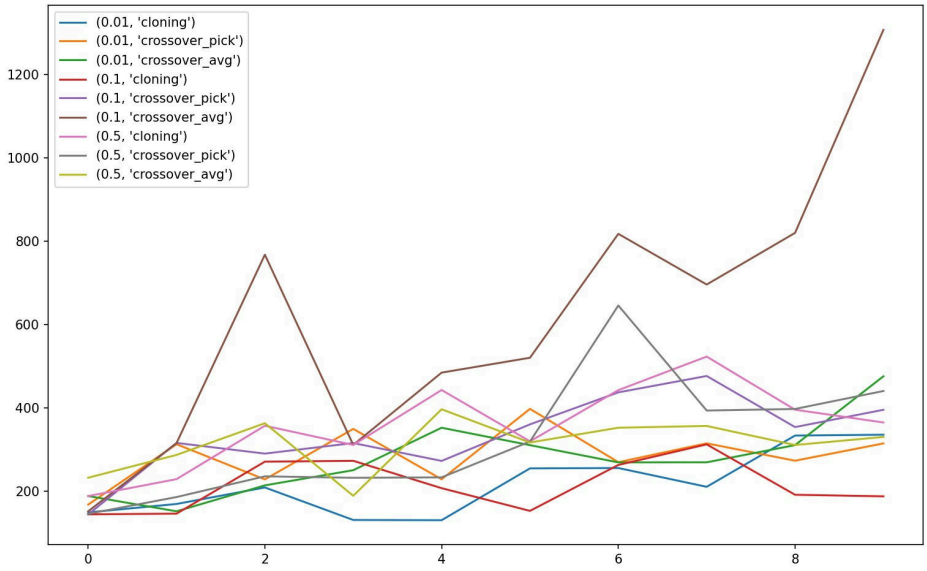

Figure 5: Results of the parameter variation study